



Spring 2025
COMP 201
Lab 8
Pre-lab Assignment
Due: Friday, 26 Dec 2025 at 09:59 AM

1 Overview

The main goal of this exercise is to help you understand how to write cache-friendly code. To achieve this, you'll use `callgrind` as a profiling tool to evaluate cache performance.

You have been provided with a simple program that computes the sum of elements in a square matrix and reports the execution times of two different methods. The first method sums the elements by rows, and the second method sums the elements by columns.

You can find the code in `main.c` in your assignment folder. It's a good idea to review it briefly before proceeding.

2 Instructions

To get started, compile the program with debugging information enabled. A Makefile is provided, so you can simply run:

```
$ make
```

Then execute the program using `valgrind` with the following command:

```
$ valgrind --tool=callgrind --simulate-cache=yes ./main
```

The results will be saved in files named `callgrind.out.PID`, where `PID` is the process identifier.

You can view the output with:

```
$ callgrind_annotate --auto=yes callgrind.out.PID
```

Important: For this exercise, focus primarily on analyzing cache performance and counting instruction segments.

3 Questions

The following questions are to be answered in `result.txt`.

1. TIO Breakdown: Assume a system with the following properties:

- Cache Size: 256 KB
- 32-bit byte-addressable address space
- 4-way Set-Associative
- Block Size: 32 bytes

Find the number of bits used for:

- Tag
- Index
- Offset

2. Given a system with L1, L2 caches, main memory, and a disk, with the following parameters:

- L1 Access Time: 2 ns
- L1 Miss Rate: 4%
- L2 Access Time: 10 ns
- L2 Miss Rate: 1%
- Main Memory Access Time: 100 ns
- Main Memory Miss Rate: ?%
- Hard Disk Access Time: 4,000,000 ns
- Hard Disk Miss Rate: 0%

Question: What should be the miss rate for the Main Memory to ensure that the system's effective access time (EAT) is faster than 5 ns?

3. Using the data from `callgrind.out.PID`, evaluate the program's cache utilization:

- What is the L1 cache miss rate for data reads and writes across the entire program?

4. Report the following:

- (a) Number of L1 cache read misses in `sumRows` and `sumColumns`.
- (b) Number of L1 cache write misses in both functions.
- (c) Number of L2 cache write misses in both functions.
- (d) Briefly explain why `sumColumn`'s L1/L2 miss behavior changes compared to `sumRows`.

5. How would cache performance be affected if `long` were used as the matrix data type instead of `int`? (No need to use `callgrind`.)
6. Based on `callgrind.out.PID`, what do the terms `Ir`, `Dr`, and `Dw` represent?

4 Submission

1. Create an `honor_code.txt` file by running the following command in your terminal. This file should include your name and a declaration that the work was completed individually:

```
$ echo "I, [Your Full Name], hereby declare that I have completed this  
assignment individually, without receiving any unauthorized assistance."  
> honor_code.txt
```

Replace `[Your Full Name]` with your actual name before running the command.

2. Compress your `result.txt`, `honor_code.txt`, and `callgrind.out.PID` files into a single zip file using the following naming convention:

```
prelab8_<yourKUusername>.zip
```

Replace `<yourKUusername>` with your actual KU username. Ensure that the zip file contains only the required files and no additional directories or compiled files.

3. Upload the compressed file to the Learn.

5 Additional Resources

As an additional resource, I **strongly recommend** watching the University of Washington's *Memory & Caches* videos (Videos 39–44 in the playlist). They give a explanation of cache fundamentals; including TIO breakdown, set associativity, locality, and how misses affect performance, which can be helpful solving this assignment.

University of Washington: Memory & Caches (Videos 39–44)